

FINFLOW AI

Financial Operating System for SMBs & Enterprises

A modular monolithic core built with Java 21, Spring Boot 3.4, Spring Security, PostgreSQL, and React. Designed with ACID properties, pessimistic locking, hierarchical approval trees, and granular audit log serialization.

1. Executive Summary & Project Vision

FinFlow AI is a production-grade Financial Operating System engineered to address operational capital leakage in Small and Medium Businesses (SMBs). Unlike typical basic CRUD expense trackers, FinFlow AI serves as an ERP-lite platform that manages the entire lifecycle of corporate spend: from organization bootstrap and role-based directory management to departmental budget allocations, vendor invoice tracking, multi-tier expense approval trees, and reimbursement payout disbursements.

The platform enforces corporate compliance by automating control checks at every stage. Budgets cannot be overrun, and approval routing dynamically shifts depending on risk thresholds. Every action that modifies the system's financial state is permanently logged to an immutable audit ledger, complete with before-and-after JSON diff tracking.

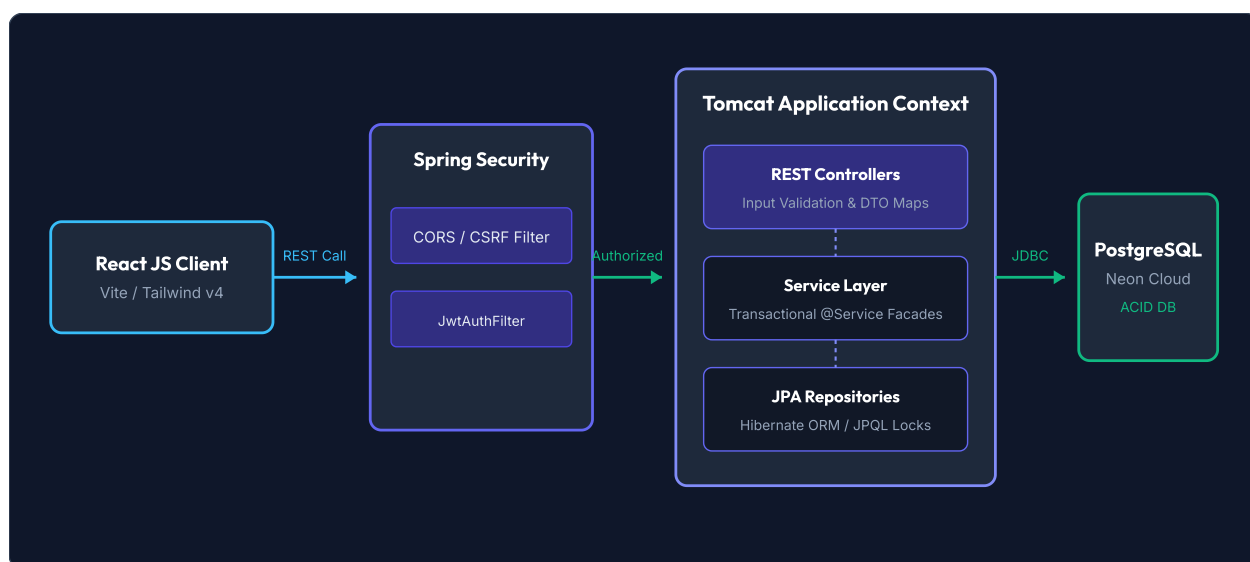
Core Business Value Propositions

- **Preventative Budgeting:** Eliminates end-of-month surprises by running real-time, transaction-locked check validation on departmental budgets before expenses can enter the approval stream.
- **Role Segregation & Compliance:** Ensures division of duties by mapping strict role permissions across Employees, Managers, Finance Managers, Auditors, and Admins.
- **Automated Risk Routing:** Streamlines simple workflows (claims under \$5,000 auto-approve) while guaranteeing multi-tier manual evaluation for medium to high-value capital outflows.
- **Audit Readiness:** Creates a continuous, tamper-proof record of organizational actions for compliance review.

2. System Architecture & Request Flows

FinFlow AI is architected as a **Modular Monolith**. By structuring the codebase along feature-based domain boundaries (modules), the system maintains compile-time isolation between contexts while avoiding the network complexity, latency, and distributed transaction challenges of microservices in its early phases.

System Request & Filter Pipeline Diagram



The request lifecycle operates as follows:

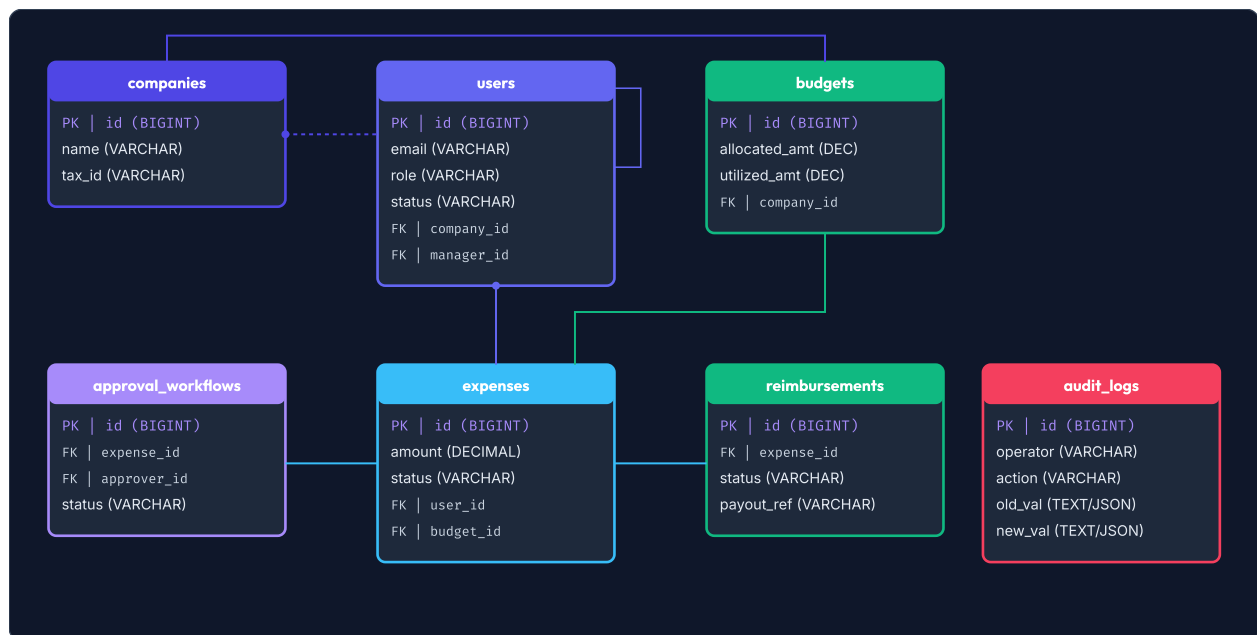
1. The client issues a REST query to port 8080 under the `/api` prefix (which maps to the Tomcat context path).
2. The request hits the Spring Security Filter Chain. `CORS` policies are evaluated, and `CSRF` settings are checked.
3. The `JwtAuthenticationFilter` extracts the Bearer token from the `Authorization` header, verifies the cryptographic signature using the `HS512` JWT token provider, deserializes the user credentials/authorities, and binds them to the thread-local `SecurityContextHolder`.
4. Once passed, Spring MVC inspects request maps and passes control to the correct `@RestController`. Parameter validators (Jakarta bean validation annotations like `@NotNull` and `@Size`) execute.

5. The controller calls the domain service inside a transactional context (managed via `@Transactional` proxy boundaries). In this layer, business rules are evaluated.
6. Services retrieve/update data via `JpaRepository` interfaces. Queries execute on the PostgreSQL database instance hosted in Neon, maintaining ACID guarantees.

3. Relational Schema & Entity Relationships

The relational database design utilizes clean normalization strategies to preserve data integrity while mapping real-world business hierarchies.

Database Entity Relationship Diagram (ERD)



4. Module Walkthroughs

FinFlow AI is structured dynamically across 12 distinct logical modules. Here is an overview of how the frontend UI page views connect with the backend service layer implementations:

Module / Domain	Backend Architecture & Responsibilities	Frontend Views & Client State
1. Auth	Handles stateless credentials validation via <code>AuthController</code> . Generates short-lived JWT access tokens and long-lived database-backed refresh tokens. Prevents session hijacking.	Premium Glassmorphic Login & Register Views. Operates with an <code>AuthContext</code> provider, holding active session states and user metadata.
2. Company	Manages organizational profiles (name, tax ID, addresses). Serves as the base tenant barrier.	Administration Settings panel. Handles tenant metadata queries and controls modifications.
3. User & RBAC	Enforces Role-Based Access Control policies. Allows admins to invite users, modify statuses (active/inactive), and change user roles.	Company Directory table. Offers search/filtering and actions (such as deactivate, change role) protected behind client-side user role flags.
4. Budgets	Manages departmental funding structures. Validates date-range overlaps, deducts funds during approval, and throws exceptions on overrun.	Budget Dashboard. Displays tracking charts mapping allocated vs. utilized funds, highlighting low-balance alerts.
5. Vendors	Stores external suppliers profiles. Enforces referential validation to block supplier deletion if linked to unpaid invoices.	Vendors Management View. A modal-driven creation form and directories tracking vendor status.

Module / Domain	Backend Architecture & Responsibilities	Frontend Views & Client State
6. Invoices	Manages accounts payable bills. Files PDF files locally, tracks due dates, and alerts users of overdue accounts.	Invoice Ledger page. Shows bills status, highlights overdue items, and integrates file uploads.
7. Expenses	Core expense tracker. Saves claims as drafts, attaches files locally, and manages expense transitions.	My Expenses Dashboard. Supports drag-and-drop receipt uploads and lists claims in tabs grouped by status.
8. Approvals	Workflow manager routing claims dynamically. Auto-approves under \$5,000; routes to manager for \$5,000–\$25,000; requires Finance Manager pool for higher values.	Review Inbox. Displays incoming approval cards for managers, with quick actions to approve, reject, or comment.
9. Reimbursements	Logs employee payouts. Links approved expenses to pending reimbursements and records payment references.	Disbursements Tracker. Provides Finance Managers a payment action view, complete with payment reference input.
10. Audit Logging	Captures system modifications (action, user, changes, timestamp). Serializes old and new entity states to JSON.	Audit log viewer. Displays log events in tables, showing detailed JSON differences.
11. Dashboard	Aggregates statistics (pending claims, utilized budget percentages, due invoices) into single payloads.	User Dashboard Hub. Displays modern metric cards, progress indicators, and glassmorphic stats.
12. Analytics	Aggregates metrics for charting (monthly spend trends, departmental distributions).	Analytics Hub. Uses Recharts to render interactive line, bar, and pie graphs.

5. Complete REST API Endpoint Directory

The backend exposes standardized JSON endpoints secured with method-level authorization checks.

Verb	Endpoint Path	Required Role	Request Body DTO
POST	/api/auth/register	Public	RegisterRequest
POST	/api/auth/login	Public	LoginRequest
POST	/api/auth/refresh	Public	RefreshTokenRequest
POST	/api/users/invite	ADMIN , FINANCE_MANAGER	InviteEmployeeRequest
PUT	/api/users/{id}/activate	ADMIN , FINANCE_MANAGER	None
PUT	/api/users/{id}/deactivate	ADMIN , FINANCE_MANAGER	None

Verb	Endpoint Path	Required Role	Request Body DTO
POST	/api/budgets	ADMIN , FINANCE_MANAGER	BudgetRequest
GET	/api/budgets/active	Authenticated	None
POST	/api/expenses	Authenticated	Multipart: DTO &
POST	/api/expenses/{id}/submit	Owner Only	None
GET	/api/expenses/review	MANAGER , FINANCE_MANAGER	None
PATCH	/api/approvals/{id}/approve	Approver Only	ApprovalReques
PATCH	/api/approvals/{id}/reject	Approver Only	ApprovalReques

Verb	Endpoint Path	Required Role	Request Body DTO
PATCH	/api/reimbursements/{id}/pay	FINANCE_MANAGER	PaymentRequest
POST	/api/invoices	ADMIN , FINANCE_MANAGER	Multipart: DTO &
GET	/api/audit-logs	ADMIN , AUDITOR	None

6. Core Design Patterns & Technical Deep Dive

1. Concurrency Control (Preventing Budget Double-Spending)

In high-concurrency enterprise applications, two managers approving expenses simultaneously could read the same budget database row, verify that `utilizedAmount + claimAmount ≤ allocatedAmount`, and write conflicting updates. This race condition leads to budget overruns.

To prevent this, FinFlow AI implements **Pessimistic Write Locking** during budget utilization. The repository method is declared as:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT b FROM Budget b WHERE b.id = :id")
Optional<Budget> findBudgetForUpdate(@Param("id") Long id);
```

When transaction A executes this query, Hibernate issues a `SELECT ... FOR UPDATE` SQL statement. This locks the database row. When transaction B attempts to retrieve the same budget, it blocks at the database level until transaction A completes its commit or rollback, ensuring strict database isolation and consistency.

2. DTO Pattern & MapStruct Code Mappings

Exposing JPA entity classes directly to the web layer presents several issues: it leaks password hashes, creates lazy initialization exceptions on un-fetched relations, and couples API contracts to database changes.

To address this, we use clean DTO boundaries mapped via **MapStruct**. MapStruct works during the Maven compilation phase, generating type-safe, reflection-free Java code:

```
@Mapper(componentModel = "spring")
public interface UserMapper {
    @Mapping(target = "companyId", source = "company.id")
    @Mapping(target = "managerId", source = "manager.id")
    @Mapping(target = "managerName", expression = "java(user.getManager() != null ? user.
    UserResponse toResponse(User user);
}
```

This compilation approach performs mapping using native getters and setters, eliminating the performance overhead of runtime reflection library engines (like ModelMapper).

3. Global Rest Exception Handler

Centralizing exception handling in a `@RestControllerAdvice` class keeps code clean and ensures clients receive predictable error payloads:

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ApiResponse<Void>> handleBusiness(BusinessException ex) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(ApiResponse.error(ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResponse<Void>> handleValidation(MethodArgumentNotValidException ex) {
        String details = ex.getBindingResult().getFieldErrors().stream()
            .map(error -> error.getField() + ": " + error.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY)
            .body(ApiResponse.error("Validation failed: " + details));
    }
}
```

7. Scaling & Production Architecture Roadmap

To transition FinFlow AI from a monolith to a high-volume enterprise application, we have planned a microservices migration path:

Asynchronous Logging (Kafka)

Writing audit logs synchronously blocks transaction threads, slowing down key database operations.

We will introduce a Kafka publisher in the service layer. Instead of writing directly to the database, audit details will be published to a `finflow-audit-topic`. An audit listener consumer will read these messages asynchronously and batch-write logs to the database, isolating logging from business transactions.

Distributed Caching (Redis)

Stateless JWT verification requires database lookups for user directory access, which degrades database performance.

We will integrate Redis to cache active budget records, user profiles, and organization metadata. Redis will also manage a JWT blacklist cache for active session invalidations on logout, and coordinate distributed locks via Redisson for budget deductions.

Dockerized Deployment Setup

To ensure consistent deployment across environments, multi-stage Docker builds are planned:

```
# Multi-Stage Backend Dockerfile
FROM maven:3.9-eclipse-temurin-21-alpine AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/finflow-ai-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

8. High-Yield Interview Preparation Q&A Guide

This guide compiles 25 detailed interview questions and answers directly related to the design and implementation of FinFlow AI.

Q1: What is a modular monolith, and why did you choose it over microservices for Phase 1?

Answer: A modular monolith is an architecture where code is organized in self-contained packages aligned by domain features rather than horizontal technical layers. We chose this approach for Phase 1 to avoid the operational overhead of microservices (like network latency, deployment complexity, and distributed tracing). Keeping all features within a single deployment unit makes transaction management straightforward while feature-based packaging preserves clear boundaries for future extraction.

Q2: Explain how Pessimistic Write Locking works in your application. Which database query does it generate?

Answer: We use `@Lock(LockModeType.PESSIMISTIC_WRITE)` on budget queries to handle concurrent expense claims. This directs the database to acquire a lock on the queried row:

```
SELECT id, allocated_amount, utilized_amount FROM budgets WHERE id = ? FOR UPDATE;
```

Any concurrent transaction trying to read or write to this budget row must wait until the locking transaction commits, preventing budget utilization race conditions.

Q3: What are the differences between optimistic and pessimistic locking? When would you use each?

Answer: Optimistic locking uses a version field (e.g. `@Version`) and checks if the version has changed before writing updates. If it has, it throws an exception. This works well for read-heavy systems with low collision rates. Pessimistic locking acquires a database lock on the row, blocking other transactions. This is suited for write-heavy, high-collision scenarios (like budget deductions or stock balances) where resolving conflicts via rollbacks would be inefficient.

Q4: How did you implement JWT refresh tokens? What vulnerability does this address?

Answer: Access tokens expire quickly (e.g., 20 minutes) to limit exposure if compromised. Refresh tokens are long-lived and stored securely in the database. When the access token expires, the client sends the refresh token to the `/api/auth/refresh` endpoint to acquire a new access token, allowing users to remain logged in securely.

Q5: What is the purpose of the axios interceptor on the React frontend?

Answer: The frontend Axios client implements response interceptors. If a request fails with a 401 Unauthorized error (indicating an expired access token), the interceptor pauses outgoing requests, calls `/auth/refresh` to refresh the token, updates client headers, and retries the failed request.

Q6: How does Spring Security authenticate users? Walk through the filter chain.

Answer: The filter chain intercepts incoming requests. The `JwtAuthenticationFilter` checks for a Bearer token in the `Authorization` header. If found, it validates the token signature, extracts the username and roles, creates a `UsernamePasswordAuthenticationToken` containing the user's authorities, and stores it in the thread-local `SecurityContextHolder`, marking the user as authenticated.

Q7: Why do we use the DTO (Data Transfer Object) pattern?

Answer: DTOs separate database models from API contracts. This decouples client payloads from schema changes, prevents lazy initialization exceptions when serializing entity relationships, and keeps sensitive fields (like password hashes) out of API responses.

Q8: How does MapStruct compare to runtime reflection mappers like ModelMapper?

Answer: MapStruct generates standard Java mapping code during compile time, ensuring type safety and optimal performance. Reflection-based mappers evaluate and convert fields at runtime, which incurs a performance cost.

Q9: Explain the ACID properties of database transactions and how PostgreSQL enforces them.

Answer:

- **Atomicity:** Enforces that all operations in a transaction commit successfully, or all rollback.

- **Consistency:** Ensures transactions transition the database from one valid state to another, preserving constraints.
- **Isolation:** Uses transaction isolation levels to keep concurrent operations from interfering.
- **Durability:** Guarantees committed updates are persisted, even in a system crash (typically using write-ahead logs).

Q10: What are dirty reads, non-repeatable reads, and phantom reads?

Answer:

- **Dirty Read:** A transaction reads uncommitted changes from another transaction.
- **Non-repeatable Read:** A transaction reads a row, and a concurrent transaction updates it, so re-reading the row returns different data.
- **Phantom Read:** A transaction queries rows matching a condition, and a concurrent transaction inserts new matching rows, so re-running the query returns new "phantom" rows.

Q11: What is the default transaction isolation level in PostgreSQL? How does it compare to Read Committed?

Answer: The default level is Read Committed. In this level, a query only sees data committed before the query began. It prevents dirty reads, but non-repeatable and phantom reads can still occur.

Q12: How does the Global Exception Handler work in Spring Boot?

Answer: An exception handler uses `@RestControllerAdvice` to catch exceptions thrown by controller actions. It intercepts the exception, formats it into a standardized `ApiResponse` payload, and sets the appropriate HTTP status code.

Q13: Why did you implement manual audit logging instead of Hibernate Entity Listeners?

Answer: Hibernate listeners run inside entity lifecycle phases, which can trigger unexpected flush events, cause session issues, or lock transactions. Implementing manual logging in the service layer gives us precise control over when logs are created and allows us to serialize before/after states using Jackson's `ObjectMapper` without database hook conflicts.

Q14: Explain the dynamic routing logic of the expense approval engine.

Answer: The routing engine uses the claim amount to determine the approval path:

- **Under \$5,000:** Auto-approves and generates a pending reimbursement.
- **\$5,000 to \$25,000:** Routes to the employee's direct manager.
- **Over \$25,000:** Routes to the Finance Manager approval pool.

Q15: How does the application prevent vendor records from being deleted if they have unpaid invoices?

Answer: Before deleting a vendor, the system calls `invoiceRepository.existsByVendorIdAndStatusNot(vendorId, InvoiceStatus.PAID)`. If unpaid invoices exist, it throws a business validation exception to protect relational data.

Q16: What is a lazy initialization exception, and how does your DTO setup prevent it?

Answer: This exception occurs when the system attempts to access a lazily-loaded entity relationship outside of an active database session. Because DTO conversion occurs within the service layer transaction context (while the Hibernate session is active), relations are fetched and converted to flat values safely before the session closes.

Q17: How does Spring Boot's IoC container manage dependency injection?

Answer: The IoC (Inversion of Control) container manages bean instantiation, configuration, and lifecycles. Dependencies are injected using constructor injection, which makes components easier to unit test.

Q18: What is the benefit of the StorageService abstraction for file uploads?

Answer: The application depends on a generic `StorageService` interface. During development, it uses a local file implementation. For production, we can swap in an AWS S3 implementation without changing any business logic.

Q19: How does the application validate dates when creating department budgets?

Answer: Before saving a budget, the system runs a query to verify there are no active budgets for the same department with overlapping dates, ensuring clean funding windows.

Q20: What is the benefit of using an API Gateway during a microservices migration?

Answer: An API Gateway serves as a single entry point for clients, routing requests to the appropriate microservices, managing CORS, handling centralized rate limiting, and

verifying JWT tokens to shield downstream services.

Q21: How would you scale audit logging using Kafka?

Answer: We would publish audit logs to a Kafka topic. An audit log microservice would consume these messages asynchronously, saving them to a dedicated database. This isolates logging from core business transactions.

Q22: How does Redis manage JWT invalidation on user logout?

Answer: When a user logs out, their JWT is added to a Redis blacklist cache with an expiration time equal to the token's remaining validity. The security filter checks this cache for every request, rejecting blacklisted tokens.

Q23: What is the purpose of multi-stage Docker builds?

Answer: Multi-stage builds use separate build and runtime environments. In the build stage, the application is compiled using Maven. The final image only copies the compiled JAR, keeping the deployment image small and secure.

Q24: How would you deploy FinFlow AI to a cloud environment like AWS?

Answer: We would package the application as Docker containers, host the database on Amazon RDS, and run the backend on ECS or EKS. Static assets would be served from S3 via CloudFront.

Q25: How do you verify user roles on the React frontend securely?

Answer: The client reads the user's role from the signed JWT payload. While client-side checks can hide UI elements, all business actions are secured on the backend using `@PreAuthorize` annotations to prevent authorization bypasses.